



Projeto final - Documento de requisitos

Bank Secure

Java com JDBC, JUnit e Mockito

Premissas

- O grupo terá duas semanas para concluir o projeto.
- Os mockups apresentados são meramente ilustrativos podem ser utilizados como base mas em termos de coloração, arredondamento e outros efeitos estéticos fica a critério do grupo;
 - **Os participantes devem desenvolver um único projeto,** considerando um backend em Java.
- Os participantes podem apresentar alternativas de solução que atendam aos requisitos demandados pela área de negócio.
- Os requisitos **essenciais são obrigatórios.**

Visão do produto

O projeto **Bank Secure** é um sistema de gerenciamento de apólices e propostas de seguros. Ele permite que os funcionários do banco (administradores) cadastrem e gerenciam os tipos de seguros que o banco oferece (Ex: Vida, Residencial, Auto), realizem cotações e registrem a venda das apólices. Clientes (usuários anônimos) podem visualizar no console os seguros disponíveis.

Funcionalidades

Há telas que agrupam as funcionalidades do produto:

- **Menu de Login** (RF02)
- **Menu de Gestão de Seguros** (RF01)
- **Menu de Cotação** (RF06, RF07)
- **Menu de Clientes** (RF04, RF05)
- **Menu de Apólices** (RF08, RF09)
- **Dashboard** (RF10)

Exemplo de simulação da Tela de Login no Terminal:

```
=====
  BankSecure - Login
=====
1. Entrar
2. Ver Seguros Disponíveis (Anônimo)
3. Sair
Escolha uma opção: 1

Usuário: [entrada do usuário]
Senha: [entrada do usuário]

[Validação do RF02 ocorre aqui]
```

Perfis e utilização

Há um único perfil que podem interagir com o sistema:

- **Usuário logado:** mantém os cadastros de Tipos de Seguros e Clientes, realiza Cotações, registra a venda de Apólices e acessa o Dashboard (simulado no console).
- **Usuário Anônimo:** pode listar os Tipos de Seguros disponíveis e suas características básicas.

Requisitos funcionais (casos de uso)

[RF01] Gestão de Tipos de Seguros

Prioridade: Essencial

O sistema deve permitir que um usuário logado **cadastre, altere e exclua** Tipos de Seguros (Ex: *Seguro Auto*, *Seguro Residencial*). Um Seguro deve ter: Título (String), Cobertura Mínima (String), Valor de Prêmio Base (Decimal).

[RF02] Efetuar Login

Prioridade: Essencial

O sistema deve permitir que um funcionário efetue login com usuário e senha (simulação simples de autenticação no Java/Console).

[RF03] Validação de Seguro

Prioridade: Essencial

Ao cadastrar/alterar um Seguro, o sistema deve validar as seguintes regras no Java (antes do JDBC):

- **Título:** Campo obrigatório.
- **Valor de Prêmio Base:** Campo obrigatório e deve ser um número decimal positivo.

[RF04] Gestão de Clientes

Prioridade: Essencial

O sistema deve permitir o **cadastro** e a **listagem** de Clientes. Um Cliente deve ter: Nome, CPF, Data de Nascimento.

[RF05] Regra de Elegibilidade (Idade)

Prioridade: Essencial

Ao cadastrar um Cliente, o sistema deve **validar** no Java: O cliente deve ser **maior de 18 anos** com base na Data de Nascimento. Caso contrário, a operação de cadastro deve ser impedida.

[RF06] Realizar Cotação

Prioridade: Essencial

O sistema deve permitir que o funcionário selecione um **Cliente** e um **Tipo de Seguro** para realizar o cálculo e exibição do **Valor do Prêmio Final** (RF07).

[RF07] Regra de Precificação (Cálculo)

Prioridade: Importante

O **Valor do Prêmio Final** da cotação deve ser calculado no Java, aplicando as regras:

1. **Valor Inicial:** Valor de Prêmio Base do Seguro.
2. **Taxa padrão:** Adicionar **5%** sobre o Valor de Prêmio Base.
3. **Bônus por Idade:** Se o cliente for **maior de 60 anos**, aplicar um **acréscimo** de **R\$ 100,00** (valor fixo).
4. **Fator Risco:** Multiplicar o prêmio pela **Taxa Fixa de Risco de 1.10** (simulando um fator de mercado, valor definido no código Java).

[RF08] Registro de Apólice (Venda)

Prioridade: Essencial

Após a cotação (RF06/RF07), o funcionário deve poder **registrar a venda** (gerar a apólice) persistindo no H2: Cliente, Seguro, Valor Final, Data de Início da Vigência (data atual) e Data de Fim da Vigência (data atual + 1 ano).

[RF09] Renovação de Apólice

Prioridade: Desejável

O sistema deve ter um menu para listar as Apólices a vencer (vigência de 1 ano) e permitir a **renovação** de uma apólice (criando uma nova Apólice com o Valor Final recalculado na hora e uma nova vigência de 1 ano futura).

[RF10] Dashboard (Sumário Simples)

Prioridade: Essencial

O sistema deve possuir um "Dashboard" (exibido no console) que exiba um sumário dos seguros vendidos, totalizando a **quantidade de apólices** vendidas e o **valor total arrecadado** por **Tipo de Seguro**.

Classes Mínimas Sugeridas (Orientação a Objetos)

Para atender aos requisitos, os alunos deverão criar, no mínimo, as seguintes classes:

- **Cliente**
- **Seguro**
- **Apolice**
- **Funcionario** (para o login)
- **CotacaoService** (Classe de Serviço para implementar o RF07)
- **ClienteDAO / SeguroDAO / ApoliceDAO** (DAO - Data Access Object - para a lógica JDBC pura)
- **App** (Classe principal para o Menu/Interface no Terminal)

Requisitos não funcionais

[NF01] Tratamento de Exceções de Persistência

Prioridade: Essencial

O sistema deve implementar o tratamento adequado das exceções de SQL (**SQLException**) em todas as operações de banco de dados (JDBC), garantindo que:

1. O fluxo principal da aplicação **não seja interrompido** abruptamente em caso de erro de banco.
2. Mensagens de erro **amigáveis** e claras sejam exibidas no console ao usuário (em vez de *stack traces* genéricos).
3. O recurso de conexão com o banco de dados seja **sempre fechado** (**Connection.close()**), mesmo em caso de exceção (utilização do bloco **try-catch-finally** ou *try-with-resources*).

[NF02] Cobertura de Testes Unitários

Prioridade: Essencial

O projeto deve atingir uma **cobertura mínima de 50%** (cinquenta por cento) para as classes que contêm a Lógica de Negócio (Service/Domínio), utilizando JUnit para testes unitários e Mockito para simulação (Mocks).

O aluno deve apresentar o relatório de cobertura com JaCoCo.

Restrições arquiteturais

Frameworks e bibliotecas

O projeto **backend** deve utilizar as seguintes tecnologias:

- Java
- JDBC
- H2
- JUnit 5
- Mockito

Padrões e design

A codificação deve seguir uma estrutura de organização de código que promova a separação de responsabilidades (princípio *Single Responsibility Principle*).

Os alunos devem organizar o código em pacotes lógicos no Java:

Pacote Sugerido	Responsabilidade	Classes Exemplo
br.com.banksecure.app	Contém a classe principal (<code>main</code>) e a lógica de interação com o	App

	Terminal/Console (Menus e I/O).	
br.com.banksecure.domain	Contém as classes que representam as entidades e o modelo de negócios. Não deve ter lógica de persistência.	Cliente, Seguro, Apolice, Funcionario
br.com.banksecure.service	Contém a Lógica de Negócio (Regras de Negócio). Deve ser a camada para ser testada pelo RNF02 (JUnit). Não deve ter código JDBC.	CotacaoService, ClienteService, ApoliceService
br.com.banksecure.infra.dao	Contém a Lógica de Persistência (JDBC Puro). É aqui que o acesso ao H2 e o tratamento do <code>SQLException</code> (NF01) devem ocorrer.	ClienteDAO, SeguroDAO, ApoliceDAO
br.com.banksecure.infra.db	Contém a classe responsável por gerenciar a conexão com o SGBD H2.	ConnectionFactory